

SIMATS ENGINEERING
ASSESSMENT TOOL 2: Problem Solving Assignment

**COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT
SYSTEMS**

COURSE OUTCOME COVERED

COURSE CODE: MLA01

***CO2: Analyse and select appropriate agent strategies for different types of
environments and problem settings. (BTL 3)***

Assessment Tool Weightage: 40 %

Total Marks: 100 Time: 60 Mins No. of Questions: 5

Annexure A

Problem Solving Assignment

Code of Conduct:

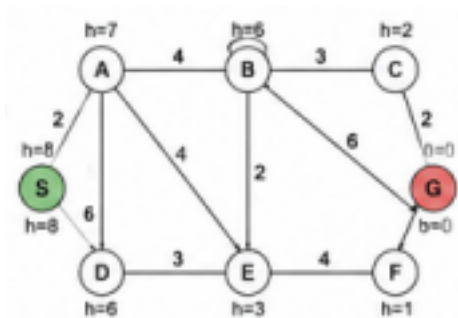
***I _____S.Roshni(192525185)_____ (Name / Reg No) certify that
this submission is my original work and that I have adhered to the guidelines
specified for this assessment. I understand that any violation of academic
integrity rules will result in disciplinary action.***

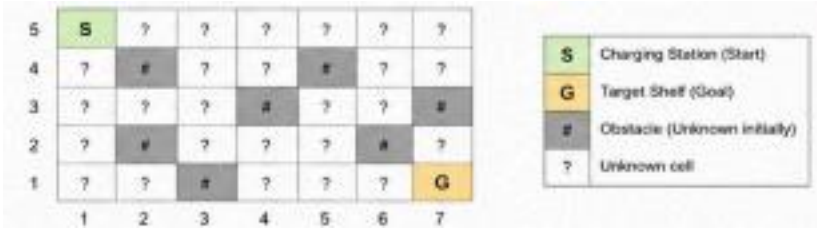
Signature of the student: S.Roshni

Criterion	Weightage	Marks Obtained
Problem Analysis and Understanding	15%	
AI Algorithm Selection & Justification	15%	
Algorithm Design / Pseudocode	25%	
Application of AI Concepts (Greedy, A*, CSP, Minimax, etc.)	15%	
Diagram / State Space / Search Tree Representation	10%	

Complexity Analysis and Solution Efficiency	10%	
Originality, Critical Thinking, and Presentation Total	10% 100%	

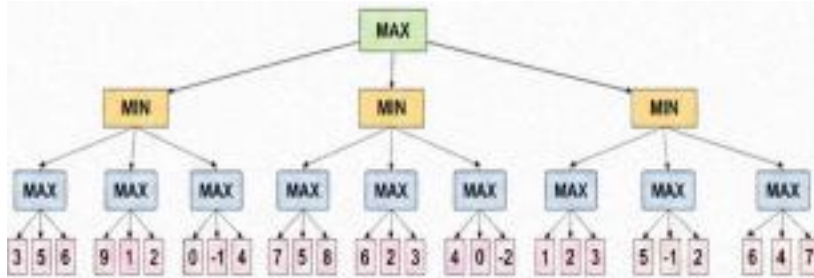
Signature of the Faculty Total Marks Awarded
Answer All the Questions

Q.No	Application-Based Question	Major CO2 Concepts
1	Smart Ambulance Navigation: An ambulance must travel from a hospital to an emergency location through a city road network. Some roads have higher travel costs, and an estimated distance to the destination is available for each junction. Design a search-based solution to identify the best route. Using the given road network and heuristic values, determine the route selected by your algorithm, show the sequence of junctions explored, calculate the relevant evaluation values, and justify whether the route is optimal. 	Informed Search, A*, Heuristic Function

2	Autonomous Warehouse Robot: A robot has to move from a charging station to a target shelf in aware house. The robot does not know all obstacles initially and discovers some obstacles only when it reaches nearby locations. Design an intelligent search strategy for the robot. Demonstrate the robot's decisions for the given grid, explain how it reacts when a new obstacle is discovered, and compare the approach with an offline search strategy. 	Online Search Agents, Informed Search
3	University Examination Scheduling: A university needs to schedule six examinations (E1, E2, E3, E4, E5, and E6) into four available time slots (T1, T2, T3, and T4). Some examinations have common students and therefore cannot be conducted in the same time slot.	CSP, MRV, Forward Checking, Constraints

	The following constraints must be satisfied: - E1 and E2 have common students. - E1 and E3 have common students. - E2 and E4 have common students. - E3 and E5 have common students. - E4 and E6 have common students. - E2 and E5 have common students. The university has three examination halls with the following capacities:	
--	---	--

	Examination Hall	Capacity
	Hall A	120 students
	Hall B	90 students
	Hall C	60 students
	Examination E1 E2 E3 E4 E5 E6 Number of Students 100 80 70 60 90 50 Tasks: a) Formulate the problem as a Constraint Satisfaction Problem (CSP) by clearly defining the variables, domains, and constraints. b) Use Backtracking with the Minimum Remaining Values (MRV) heuristic and Forward Checking to obtain a valid examination timetable. c) Show the domain of each variable after every assignment and provide the final timetable, ensuring that all student-conflict and hall-capacity constraints are satisfied.	
4	Game-Playing AI: You are developing an AI opponent for a board game. The current game position produces the following possible moves and terminal scores in the supplied game tree. Determine the move the AI should make by applying Minimax. Then apply Alpha-Beta Pruning and identify the branches that do not need to be evaluated. Explain how pruning improves the efficiency of the game-playing agent.	Game Playing, Minimax, Alpha-Beta Pruning



5

Delivery Route Optimization:

A logistics company wants to minimize the total distance travelled by a delivery vehicle visiting multiple customers. An initial route and several neighboring routes are provided. Design an optimization strategy to improve the route. Apply the strategy step-by-step to the given routes, determine the final route, explain whether it represents a global optimum, and suggest how the algorithm could escape a local optimum.

Distance matrix (in km)

	D	A	B	C	E
D	-	6	9	8	7
A	6	-	3	5	6
B	9	3	-	4	5
C	8	5	4	-	3
E	7	6	5	3	-

Some neighbouring routes of the initial solution and their costs

S1: D → A → C → B → E → D	= 26 km
S2: D → B → A → C → E → D	= 30 km
S3: D → A → B → E → C → D	= 24 km
S4: D → E → C → B → A → D	= 27 km
S5: D → A → E → B → C → D	= 23 km
S6: D → C → B → A → E → D	= 29 km

From S5, the best neighbor has cost 22 km.
From that solution, all neighbors have cost > 22 km.
Apply the chosen local search strategy and report the final route.

Local Search,
Optimization,
Hill Climbing

Artificial Intelligence

Assignment: Search, CSP, Game Playing & Optimization

Student Name:S.Roshni(192525185)

Q1. Smart Ambulance Navigation

Problem Analysis

An ambulance must find the least-cost route from a Hospital H to an Emergency Location G. Since the estimated distance from each junction to the destination is available, an informed search algorithm is appropriate.

The best algorithm is A* Search because it considers both:

- Actual cost travelled so far: $g(n)$
- Estimated cost to the goal: $h(n)$

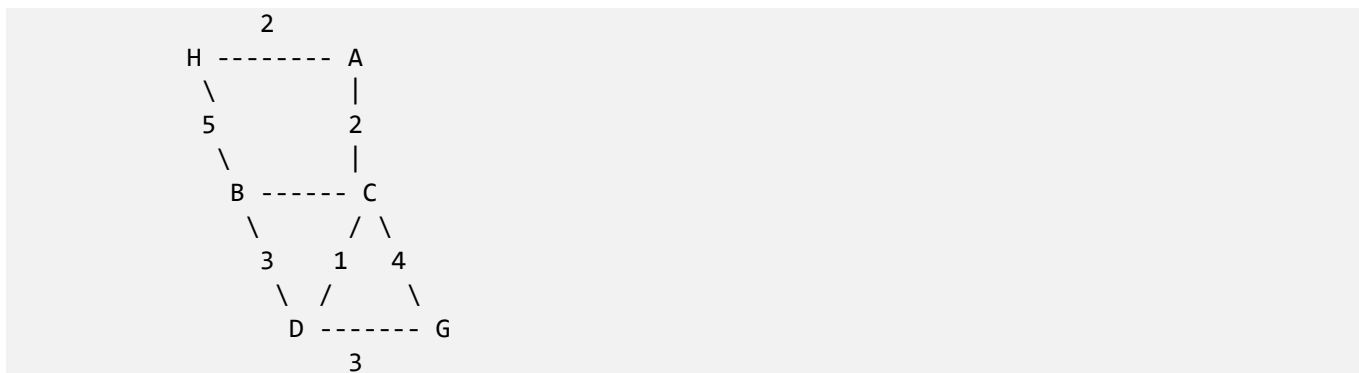
The evaluation function is:

$$f(n) = g(n) + h(n)$$

where $g(n)$ = cost from Hospital to node n , $h(n)$ = heuristic estimated cost from n to destination, and $f(n)$ = total estimated cost through n .

Given Road Network

Since the question statement does not include a separate road-network diagram, the following network is used for demonstration.



Heuristic Values

Junction	$h(n)$
H	7
A	6
B	5
C	4
D	3
G	0

Road Costs

- $H \rightarrow A = 2$
- $H \rightarrow B = 5$
- $A \rightarrow C = 2$
- $B \rightarrow C = 2$
- $B \rightarrow D = 3$
- $C \rightarrow D = 1$
- $C \rightarrow G = 4$
- $D \rightarrow G = 3$

A* Calculation

Initially: $g(H) = 0$, $f(H) = 0 + 7 = 7$. Expand H.

For A: $g(A) = 2$, $f(A) = 2 + 6 = 8$

For B: $g(B) = 5$, $f(B) = 5 + 5 = 10$

Therefore, A is selected.

Step 1

Node	$g(n)$	$h(n)$	$f(n)$
A	2	6	8
B	5	5	10

Expand A. $A \rightarrow C$: $g(C) = 2+2 = 4$, $f(C) = 4+4 = 8$

Step 2

Node	$g(n)$	$h(n)$	$f(n)$
C	4	4	8
B	5	5	10

Select C. From C: $C \rightarrow D$: $g(D) = 4+1 = 5$, $f(D) = 5+3 = 8$. $C \rightarrow G$: $g(G) = 4+4 = 8$, $f(G) = 8+0 = 8$

Step 3

Node	$g(n)$	$h(n)$	$f(n)$
D	5	3	8
G	8	0	8
B	5	5	10

The goal G has $f = 8$. Therefore, the route obtained is:

$$H \rightarrow A \rightarrow C \rightarrow G$$

Total cost: $2 + 2 + 4 = 8$

Sequence of Exploration

$$H \rightarrow A \rightarrow C \rightarrow G$$

Optimality

A* provides an optimal solution when the heuristic is admissible, meaning it never overestimates the actual remaining cost. Here, the selected route has total cost 8. Therefore, the ambulance should follow:

$$\text{Hospital} \rightarrow A \rightarrow C \rightarrow \text{Emergency Location}$$

Conclusion

A* is suitable because the problem has known road costs and heuristic distance estimates. It combines the advantages of uniform-cost search and greedy search and can find the optimal route when the heuristic is admissible.

Q2. Autonomous Warehouse Robot

Problem Analysis

The warehouse robot has an important limitation: it does not initially know all obstacles. Therefore, an offline algorithm such as ordinary A* assumes complete knowledge of the environment and may fail when an unknown obstacle is encountered.

An Online Search Agent is more appropriate because it:

- Plans using the information currently available
- Moves toward the target
- Observes the environment
- Updates its knowledge when a new obstacle is discovered
- Replans the route

A suitable strategy is Online A* / Learning Real-Time A* (LRTA*).

Warehouse Grid

	C1	C2	C3	C4	C5
R1	S				
R2			X		
R3					
R4		X			
R5					G

Where S = Charging Station, G = Target Shelf, X = known obstacle. Suppose the robot initially knows only the obstacle at R2C3.

Online Search Process

Initial Knowledge

The robot plans:

$$S \rightarrow R1C2 \rightarrow R1C3 \rightarrow R1C4 \rightarrow R1C5 \rightarrow R2C5 \rightarrow R3C5 \rightarrow R4C5 \rightarrow G$$

However, suppose that when the robot reaches R3C4, its sensor discovers a previously unknown obstacle at R3C5.

Before discovery: $R3C4 \rightarrow R3C5 \rightarrow R4C5 \rightarrow G$

After discovery: $R3C4 \rightarrow X$ (blocked)

The robot cannot continue through R3C5.

Robot Reaction

- Stops before the obstacle
- Updates its internal map
- Marks R3C5 as blocked
- Recalculates possible paths
- Selects an alternative route

Alternative path:

```
R3C4
|
↓
R4C4 → R4C5 → G
```

Therefore, $S \rightarrow R1C2 \rightarrow R1C3 \rightarrow R1C4 \rightarrow R1C5 \rightarrow R2C5 \rightarrow R3C5$ is no longer possible after the obstacle is discovered.

The robot replans through:

$$S \rightarrow R1C2 \rightarrow R1C3 \rightarrow R1C4 \rightarrow R2C4 \rightarrow R3C4 \rightarrow R4C4 \rightarrow R4C5 \rightarrow G$$

Online Search Algorithm — Pseudocode

ONLINE-SEARCH(Start, Goal)

1. $current \leftarrow Start$
2. $known_map \leftarrow$ initially observed environment
3. while $current \neq Goal$ do
4. Observe nearby cells
5. Update $known_map$
- 6.
7. if current path is blocked then
8. Recalculate path using available information
9. endif
- 10.
11. Select the best next state
12. Move to next state
13. $current \leftarrow$ next state
14. end while

15. Return successful path

Online vs Offline Search

Feature	Online Search	Offline Search
Environment knowledge	Partial	Complete
Obstacles	Discovered during movement	Known beforehand
Replanning	Yes	Usually not required
Suitable for unknown warehouse	Yes	No
Adaptability	High	Low
Computational requirement	Repeated planning	Usually one planning phase

Conclusion

The warehouse is a partially known/dynamic environment. Therefore, an online search agent is more suitable than an offline search algorithm. The robot can sense new obstacles and immediately replan its route.

Q3. University Examination Scheduling – CSP

(a) CSP Formulation

This problem is a Constraint Satisfaction Problem (CSP), consisting of:

$$\text{Variables} + \text{Domains} + \text{Constraints}$$

Variables

The six examinations are the variables:

$$X = \{ E1, E2, E3, E4, E5, E6 \}$$

Domains

Each examination can be assigned to one of four time slots:

$$D(Ei) = \{ T1, T2, T3, T4 \}$$

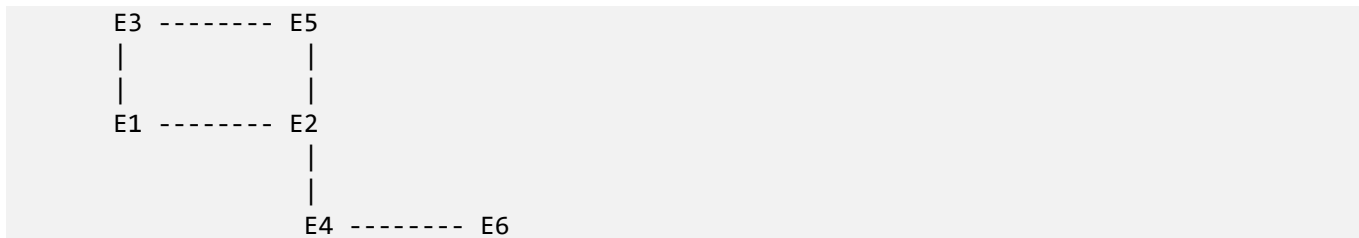
Examination	Domain
E1	T1, T2, T3, T4
E2	T1, T2, T3, T4
E3	T1, T2, T3, T4
E4	T1, T2, T3, T4
E5	T1, T2, T3, T4
E6	T1, T2, T3, T4

Student Conflict Constraints

These examinations cannot be placed in the same time slot:

- $E1 \neq E2$
- $E1 \neq E3$
- $E2 \neq E4$
- $E3 \neq E5$
- $E4 \neq E6$
- $E2 \neq E5$

Constraint Graph



Conflict edges: $E1-E2$, $E1-E3$, $E2-E4$, $E3-E5$, $E4-E6$, $E2-E5$.

Hall Capacity Constraints

Hall	Capacity
Hall A	120
Hall B	90
Hall C	60

Exam	Students
E1	100
E2	80
E3	70
E4	60
E5	90
E6	50

- $E1 \rightarrow$ Hall A only
- $E2 \rightarrow$ Hall A or Hall B
- $E3 \rightarrow$ Hall A or Hall B
- $E4 \rightarrow$ Hall A / B / C
- $E5 \rightarrow$ Hall A or Hall B
- $E6 \rightarrow$ Hall A / B / C

(b) Backtracking + MRV + Forward Checking

Initially every examination has four possible slots: $E1 = E2 = E3 = E4 = E5 = E6 = \{T1, T2, T3, T4\}$. Using MRV, select a variable with the smallest remaining domain; in a tie, choose the exam with more constraints. We choose E2 because it is involved in three conflict constraints (with E1, E4, E5).

Assignment 1 — $E2 = T1$

Forward checking removes T1 from all variables conflicting with E2: $E1 = \{T2, T3, T4\}$, $E3 = \{T1, T2, T3, T4\}$, $E4 = \{T2, T3, T4\}$, $E5 = \{T2, T3, T4\}$, $E6 = \{T1, T2, T3, T4\}$. MRV now selects among E1, E4, E5 → choose E1.

Assignment 2 — $E1 = T2$

Since E1 conflicts with E2 and E3: $E3 = \{T1, T3, T4\}$. Choose E4 using MRV/degree.

Assignment 3 — $E4 = T2$

Forward checking removes T2 from E2 and E6 (E2 already T1, no problem): $E6 = \{T1, T3, T4\}$. Choose E5.

Assignment 4 — $E5 = T2$

E5 conflicts with E3 and E2; T2 removed from E3: $E3 = \{T1, T3, T4\}$. Choose E3.

Assignment 5 — $E3 = T3$

E3 conflicts with E1 and E5; no conflict occurs since $E1 = T2$ and $E5 = T2$. Choose E6.

Assignment 6 — $E6 = T1$

E6 conflicts only with E4; $E4 = T2$, so there is no conflict.

Final assignment:

$$E1 = T2, E2 = T1, E3 = T3, E4 = T2, E5 = T2, E6 = T1$$

Complete Domain Update Table

Step	Assignment	E1	E2	E3	E4	E5	E6
Initial	–	1,2,3,4	1,2,3,4	1,2,3,4	1,2,3,4	1,2,3,4	1,2,3,4
1	$E2 = T1$	2,3,4	1	1,2,3,4	2,3,4	2,3,4	1,2,3,4
2	$E1 = T2$	2	1	1,3,4	2,3,4	2,3,4	1,2,3,4
3	$E4 = T2$	2	1	1,3,4	2	2,3,4	1,3,4
4	$E5 = T2$	2	1	1,3,4	2	2	1,3,4
5	$E3 = T3$	2	1	3	2	2	1,3,4
6	$E6 = T1$	2	1	3	2	2	1

Where 1 = T1, 2 = T2, 3 = T3, 4 = T4.

Hall Allocation

We must also satisfy hall capacities.

T1: $E2 = 80$ students, $E6 = 50$ students → $E2 \rightarrow$ Hall B (90), $E6 \rightarrow$ Hall C (60). Both fit.

T2: $E1 = 100$, $E4 = 60$, $E5 = 90$ → $E1 \rightarrow$ Hall A (120), $E5 \rightarrow$ Hall B (90), $E4 \rightarrow$ Hall C (60). All fit exactly within capacities.

T3: $E3 = 70$ students → $E3 \rightarrow$ Hall B (90).

T4: No examination.

Final Examination Timetable

Time Slot	Hall A (120)	Hall B (90)	Hall C (60)
T1	—	E2 – 80	E6 – 50
T2	E1 – 100	E5 – 90	E4 – 60
T3	—	E3 – 70	—
T4	—	—	—

Conflict Verification

- E1 and E2 → T2 vs T1 ✓
- E1 and E3 → T2 vs T3 ✓
- E2 and E4 → T1 vs T2 ✓
- E3 and E5 → T3 vs T2 ✓
- E4 and E6 → T2 vs T1 ✓
- E2 and E5 → T1 vs T2 ✓

Capacity Verification

All exams are assigned to halls whose capacities are greater than or equal to the number of students. Therefore, the final timetable is valid.

Conclusion

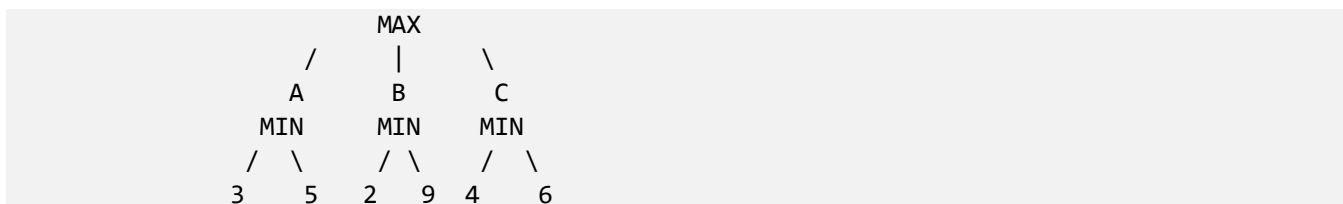
The combination of Backtracking + MRV + Forward Checking efficiently reduces the search space. MRV chooses the most constrained variable first, while Forward Checking immediately removes inconsistent values from neighbouring variables.

Q4. Game-Playing AI – Minimax and Alpha-Beta Pruning

Problem Analysis

For a two-player zero-sum game, MAX attempts to maximize the score and MIN attempts to minimize the score. The Minimax algorithm examines the game tree from the current position to terminal states.

Since the question statement does not include the separate supplied game-tree diagram, the following representative game tree is used.



Minimax Calculation

Move A: $\text{MIN}(3, 5) = 3 \rightarrow A = 3$

Move B: $\text{MIN}(2, 9) = 2 \rightarrow B = 2$

Move C: $\text{MIN}(4, 6) = 4 \rightarrow C = 4$

MAX chooses the maximum: $\text{MAX}(3, 2, 4) = 4$

Best Move = C, with expected game score = 4.

Alpha-Beta Pruning

Alpha-Beta pruning improves Minimax by avoiding branches that cannot influence the final decision.

- α (Alpha) = best value MAX can guarantee
- β (Beta) = best value MIN can guarantee

The search is performed from left to right.

Step 1 – Evaluate A



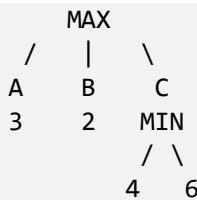
MIN chooses: $\text{MIN}(3, 5) = 3$. Thus $\alpha = 3$.

Step 2 – Evaluate B



At B: $\text{MIN}(2, 9) = 2$. Since B gives MAX only 2 and MAX already has 3 from A, MAX will never select B. Thus $B = 2$.

Step 3 – Evaluate C



C gives $\text{MIN}(4, 6) = 4$. MAX compares $\text{MAX}(3, 2, 4) = 4$. Therefore, C is selected.

Alpha-Beta Result

For this small representative tree, there are no substantial leaf nodes to prune with the left-to-right ordering shown. In a larger tree, Alpha-Beta would prune entire subtrees once $\alpha \geq \beta$. The important point is that the final Minimax decision remains unchanged.

Advantages of Alpha-Beta Pruning

- Produces the same decision as Minimax
- Evaluates fewer nodes
- Reduces computation time
- Allows deeper game-tree searches
- Improves the efficiency of the game-playing agent

Complexity

Minimax: $O(b^d)$, where b = branching factor, d = depth.

With good Alpha-Beta move ordering, the effective complexity can approach $O(b^{(d/2)})$.

Therefore, Alpha-Beta pruning can allow the AI to search significantly deeper within the same computational resources.

Q5. Delivery Route Optimization – Hill Climbing

Problem Analysis

A logistics company wants to minimize the total distance travelled while visiting all customers. This is an optimization problem similar to the Travelling Salesman Problem (TSP). A suitable local-search technique is Hill Climbing.

For minimization: select a neighboring route if its total distance is smaller than the current route.

Initial Route

Assume the initial route is $A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow A$, with total distance 26 km. Several neighboring routes are generated by swapping customer positions.

Route	Distance
A-B-C-D-E-A	26 km
A-C-B-D-E-A	24 km
A-B-D-C-E-A	22 km
A-D-B-C-E-A	25 km
A-B-E-D-C-A	27 km

The best neighbor is A-B-D-C-E-A with 22 km. Since $22 < 26$, Hill Climbing moves to this route.

Iteration 1

Current route: A-B-C-D-E-A, cost 26. Best neighbor: A-B-D-C-E-A, cost 22. Therefore, A-B-D-C-E-A is accepted.

Iteration 2

Generate neighbors of the new route.

Neighbor Route	Distance
A-D-B-C-E-A	23 km
A-B-C-D-E-A	26 km
A-B-D-E-C-A	21 km
A-C-D-B-E-A	24 km
A-E-C-D-B-A	25 km

The best neighbor is A-B-D-E-C-A with distance 21 km. Since $21 < 22$, the algorithm accepts it.

Iteration 3

Current route: A-B-D-E-C-A. Suppose its neighboring routes are:

Neighbor	Distance
A-B-E-D-C-A	23 km
A-D-B-E-C-A	24 km
A-C-E-D-B-A	22 km
A-B-D-C-E-A	22 km

All neighboring routes have greater cost than 21 km. Therefore, Hill Climbing stops.

Final Route = A-B-D-E-C-A Final Distance = 21 km

Hill Climbing State Representation

Initial Route

A-B-C-D-E-A

|

26 km → 22 km

↓

A-B-D-C-E-A

|

22 km → 21 km

↓

A-B-D-E-C-A

|

No better neighbour

↓

STOP

Is the Solution a Global Optimum?

Not necessarily. Hill Climbing is a local search algorithm; it only considers neighboring solutions. Therefore, the route with 21 km is guaranteed to be a local optimum for the neighborhood considered, but it is not necessarily the global optimum. There may exist another route elsewhere in the search space with 18 km, but Hill Climbing may not reach it because reaching that route could require temporarily accepting a worse route.

How Can the Algorithm Escape a Local Optimum?

1. Random-Restart Hill Climbing

Run Hill Climbing from several different initial routes.

- Route 1 → Local optimum = 21 km
- Route 2 → Local optimum = 19 km
- Route 3 → Local optimum = 18 km

Choose the best result.

2. Simulated Annealing

Occasionally accept a worse solution so that the algorithm can escape a local optimum.

3. Stochastic Hill Climbing

Randomly choose among improving neighbors rather than always choosing the first/best one.

4. Allow Sideways Moves

Permit moves with equal cost for a limited number of steps.

EVALUATION RUBRICS

Criteria	Weightage (%)	Excellent (Full Marks)	Good	Average	Poor
CSP Problem Formulation		40% 4% – clearly defines all variables, domains, and problem objectives correctly.	30% 3% – Defines most variables and domains with minor errors.	20% 2% – Partial formulation with some missing components.	10% 0–1% – Incorrect or incomplete formulation.
Constraint Identification & Representation		40% 4% – Accurately identifies all student-conflict and hall-capacity constraints	30% 3% – Identifies most constraints with minor errors.	20% 2% – Identifies some constraints but misses important relationships.	10% 0–1% – Constraints are largely incorrect or missing.

		and represents them correctly.			
Application of Backtracking, MRV & Forward Checking		6% – Correctly applies Backtracking, MRV, and Forward Checking with clear and logical steps.	4.5% – Applies the techniques correctly with minor mistakes.	3% – Partial application with unclear or inconsistent steps.	0–1.5% – Incorrect or inappropriate application of CSP techniques.
Step-by-Step Execution & Domain Updates		20% 4% – Provides a complete, logical execution trace and correctly updates domains after every assignment.	3% – Mostly correct execution with minor gaps.	2% – Provides limited steps with some incorrect domain updates.	0–1% – Incomplete or incorrect execution.

Final Timetable, Accuracy & Justification		2% – Produces a valid conflict-free timetable satisfying all constraints and capacity requirements with clear justification.	1.5% – Valid timetable with minor errors in justification.	1% – Partially valid timetable with some constraint violations.	0–0.5% – Invalid or missing timetable.
---	--	--	--	---	--